

Auditoría y corrección de la base de datos

Sistema de gestión — Clínica Odontológica

Documento de trazabilidad de cambios
De la versión original al esquema corregido

Base de datos	odontologia — PostgreSQL 16
Migraciones aplicadas	003_pulido_esquema.sql, 004_correcciones_ingeniero.sql
Correcciones aplicadas	más de 40, en 2 migraciones verificadas
Fecha	27/07/2026

1. Resumen ejecutivo

Este documento registra las **más de cuarenta correcciones** aplicadas a la base de datos del proyecto, desde la versión original hasta el esquema final. Reúne tres fuentes: el problema reportado sobre los nombres duplicados, una auditoría completa del esquema, y las observaciones que el ingeniero dejó en la revisión grabada.

El origen del trabajo fue un cambio a medias. Se pidió permitir segundo nombre y segundo apellido; se agregaron las cuatro columnas nuevas y se copiaron los datos, pero nunca se eliminaron las viejas ni se actualizó la aplicación. El resultado fue el mismo dato guardado dos veces, sin nada que lo mantuviera sincronizado.

La base en números

Métrica	Antes	Después	Cambio
Tablas	17	23	+6
Columnas	130	176	+46
Índices	28	70	+42
Claves foráneas	24	32	+8
Restricciones UNIQUE	10	19	+9
Restricciones CHECK	0	29	+29
Triggers	0	30	+30
Vistas	0	4	+4
Funciones	0	4	+4

Los ceros de la columna «Antes» en CHECK, triggers, vistas y funciones no son un error de medición: la base original no tenía ninguna restricción de validación, ningún trigger y ninguna vista.

Cómo se verificó cada cambio

Ninguna corrección se aplicó a ciegas. Se levantó una instancia de PostgreSQL aparte, se restauró el respaldo real de la base (18 personas, 13 usuarios, 8 empleados, 29 citas, 10 facturas, 86 registros de acceso) y se ejecutaron las migraciones sobre esa copia. Sobre ese entorno se comprobó que:

- ninguna fila se perdiera ni se alterara;
- el script de reversión devolviera el esquema exactamente al estado original, incluidos los nombres reconstruidos carácter por carácter;
- el archivo `esquema_final.sql` produjera un esquema idéntico al migrado: 176 columnas, 70 índices, 30 triggers y 4 vistas coincidentes;
- las **36 pruebas de comportamiento** pasaran, verificando que cada regla nueva bloquee exactamente lo que debe bloquear y permita lo que debe permitir.

2. El problema reportado: nombres duplicados

La tabla Persona tenía ocho columnas para almacenar un nombre. Las migraciones previas agregaron las cuatro nuevas y copiaron los datos, pero dejaron las dos originales en su lugar. Una búsqueda en todo el código del backend confirmó que las columnas nuevas **no se usaban en ningún archivo**: la aplicación seguía leyendo y escribiendo únicamente en las viejas.

Antes	Después
nombre (redundante) apellido (redundante) primerNombre segundoNombre primerApellido segundoApellido	primerNombre (obligatorio) segundoNombre primerApellido (obligatorio) segundoApellido nombreCompleto (calculada)

Por qué era un error y no solo un descuido. Guardar el mismo hecho en dos lugares sin un mecanismo que los mantenga de acuerdo garantiza que tarde o temprano se contradigan: basta con que alguien edite uno solo. La solución no fue elegir un juego de columnas y borrar el otro sin más, sino agregar nombreCompleto como **columna calculada**. PostgreSQL la deriva de las otras cuatro y no permite escribir en ella, así que es imposible que se desincronice, y el resto del sistema conserva un campo único para mostrar.

3. Primera auditoría del esquema

3.1 Tipos de datos

Columna	Antes	Después	Por qué
Cita.fecha	text	DATE	Como texto no se puede ordenar ni comparar, y nada impedía guardar «mañana»
Cita.hora	text	TIME	Mismo problema
Montos (9 columnas)	double precision	NUMERIC(12,2)	El punto flotante binario pierde centavos: el total deja de cuadrar con la suma de las líneas
Persona.fechaNac	timestamp(3)	DATE	Una fecha de nacimiento no tiene hora
rolCancela	varchar(50)	FK a Rol	Texto libre donde ya existía un catálogo

3.2 Enumerados convertidos en catálogos

Los tipos Rol y Puesto eran ENUM de PostgreSQL. Un ENUM es rígido: agregar un valor exige ALTER TYPE y quitarlo es prácticamente imposible. Además ambos guardaban el mismo hecho por duplicado, sin nada que los mantuviera de acuerdo. Se reemplazaron por tablas catálogo con significados separados: **Rol** es el nivel de acceso al sistema y **Puesto** es el cargo laboral. El ENUM EstadoCita se conservó a propósito, porque los estados de una cita son un conjunto cerrado que define la lógica de negocio, no un catálogo que el usuario administre.

Un detalle técnico que obligó a reordenar la migración: en PostgreSQL una tabla y un tipo comparten el mismo espacio de nombres, así que no se puede crear la tabla Rol mientras exista el ENUM del mismo nombre. Las columnas se convierten primero a texto, luego se eliminan los tipos, y recién entonces se crean los catálogos.

3.3 Reglas de integridad ausentes

Regla agregada	Qué permitía la base antes
UNIQUE parcial en Cita (doctor, fecha, hora)	Agendar dos pacientes con el mismo doctor a la misma hora. Es parcial para que una cita cancelada libere su horario.
UNIQUE parcial en Cita (paciente, fecha, hora)	Que un paciente estuviera en dos citas simultáneas.
UNIQUE en Persona.dni	Registrar dos veces a la misma persona. El proyecto ya había sufrido ese error.
Trigger: el doctor debe tener puesto DOCTOR	Agendar una cita con la recepcionista.
UNIQUE de correo sin distinguir mayúsculas	Que Juan@Mail.com y juan@mail.com fueran dos cuentas distintas.
FK en usuarioCancelald	Registrar una cancelación hecha por un usuario inexistente.
CHECK de montos, cantidades y formatos	Precios negativos, cantidades en cero, DNI y teléfonos con cualquier contenido.

3.4 Correcciones de comportamiento

Logs.logout tenía valor por defecto. Una sesión abierta quedaba registrada con hora de salida igual a la de entrada. Ahora queda nula hasta el cierre real.

HistorialCancelacionCita admitía un solo registro por cita. Una tabla llamada «historial» estaba limitada a una fila. Se quitó esa restricción.

CodigoVerificacion admitía un solo código por usuario. El segundo intento de verificación de un mismo usuario fallaba.

updatedAt nunca se actualizaba. Tenía valor por defecto pero no existía ningún mecanismo que lo modificara. Se agregaron triggers.

Sin borrado en cascada. Al eliminar una factura, sus líneas quedaban huérfanas.

Sin índices en claves foráneas. PostgreSQL crea el índice de la clave primaria pero no el de las foráneas. Cada JOIN recorría la tabla completa. Se agregaron 24.

4. Correcciones pedidas por el ingeniero

Las siguientes correcciones provienen de la revisión grabada. Cada una se acompaña de la cita textual que la origina.

4.1 Tabla de permisos

«Los roles está bien que vengan por un enum, pero yo prefiero una tabla de permisos, porque con la tabla de permisos yo puedo ir uno a uno cada una de las secciones del software. Cuando yo tengo un rol es bien genérico [...] Mientras que si ustedes dicen, ok, sí quiero que mire la pantalla, pero no quiero que edite nada. Entonces eso lo hago a nivel de permisos.»

Se crearon las tablas Permiso y RolPermiso, precargadas con **17 permisos** y **28 asignaciones**. El rol responde «quién sos»; el permiso responde «qué podés hacer». Los permisos se dedujeron de la descripción que el propio ingeniero hizo del alcance de cada rol, incluidas las dos exclusividades que remarcó: solo la recepcionista puede facturar y solo el doctor puede editar expedientes. La migración incluye una comprobación que aborta si alguna de esas dos exclusividades se rompe.

4.2 Campos de auditoría consistentes

«Me llama la atención porque tienen los campos de auditoría en unas tablas y en otras tablas no hay campos de auditoría [...] los campos de auditoría o van o no van.»

Faltaban en **siete tablas**. Donde ya existía una fecha equivalente no se duplicó el dato: en EspecialidadDoctor y ExpedienteDoctor la columna fechaAsociacion era exactamente un createdAt con otro nombre, así que se renombró. En el resto se rellenó con el dato real disponible: la fecha de ingreso para los empleados, la hora de entrada para los registros de acceso, la fecha de cancelación para el historial.

4.3 Nombres mnemónicos

«Si a mí me dicen que tienen un campo que se llama doctor, quiere decir que deben de tener una tabla que se llame doctores. Pero aquí lo que tengo yo en vez de doctor es un empleado. Entonces, debería llamarse empleado I.D. Así yo sé automáticamente contra qué lo voy a conectar.»

Se renombró doctorId a empleadoId en las cinco tablas que lo usaban, junto con sus claves foráneas e índices. **Una decisión que conviene explicar:** aplicada al pie de la letra, la misma regla obligaría a renombrar pacienteId a personaId, porque también apunta a Persona. No se hizo, porque Cita y Factura quedarían con dos columnas hacia la misma tabla y el nombre genérico perdería significado. La solución correcta para ese caso es una entidad Paciente propia, que quedó documentada como mejora pendiente.

4.4 Links con expiración

«Aquí lo que necesitan ustedes son links por tiempo. O links vivos o links alive [...] Yo genero un link y voy a tener una tabla donde yo voy a tener ese link por aquí y aquí le pongo cuándo expira [...] en el backend va a validar si lo que nosotros le mandamos está vivo o no.»

La tabla CodigoVerificacion ya hacía la mitad del trabajo, pero solo servía para verificar la cuenta. Se generalizó a TokenAcceso, que cubre tanto la verificación de correo como el cambio de contraseña, con fecha de expiración, marca de uso y un índice que impide que un usuario tenga dos tokens vivos del mismo tipo a la vez.

4.5 Solo se factura una cita completada

«Para que se pueda facturar una cita, esta tiene que estar en el estado completado de la cita. Si no, no se puede facturar.»

Implementado como trigger sobre Factura. Los datos existentes ya cumplían la regla, así que entró sin conflictos.

4.6 Observaciones que no requirieron cambios

El ingeniero señaló que los passwords deben estar encriptados; ya lo estaban, con bcrypt. También pidió que el puesto del empleado fuera un catálogo, cosa que ya se había hecho en la primera auditoría. Sobre la facturación, la única frase de la grabación es *«Más. La facturación. Ok. Lo demás me parece que lo llevo por buen camino»*; no dejó indicaciones específicas. Todo lo del capítulo 6 de este documento proviene de la normativa vigente, no de la revisión.

Una parte de la grabación quedó sin resolver. Al hablar de la tabla de usuarios dice *«tienen una persona ID, esto no está normalizado, desde aquí tienen cero»*, pero el audio se corta y no es posible determinar a qué se refería.

Queda además un pedido **no implementado**: el ingeniero propuso reemplazar el password temporal por una bandera de «requiere cambio de contraseña». Se detectó que el backend hace referencia a tres columnas para eso —passwordTemporal, passwordTemporalExpira y requierCambioPassword, esta última con un error de escritura— que **no existen en la base**, por lo que ese camino del login está inutilizado. La corrección quedó pendiente.

5. Segunda auditoría: redundancia restante

Una vez aplicadas las correcciones anteriores se hizo una segunda revisión completa buscando redundancia que hubiera quedado. Se encontraron cuatro casos.

5.1 Recordatorios: un grupo repetitivo

La tabla Cita tenía dos columnas booleanas, `recordatorio1h` y `recordatorio24h`: el mismo hecho con distinto parámetro. Agregar un recordatorio de 48 horas habría exigido modificar la estructura de la tabla. Eso es un **grupo repetitivo**, que es precisamente lo que la primera forma normal prohíbe. Se reemplazaron por la tabla `RecordatorioCita`, donde cada recordatorio enviado es una fila y agregar un tipo nuevo no toca el esquema.

5.2 Un dato derivado que se guardaba a mano

La columna `ExpedienteArchivo.filePath` contenía siempre `/archivos/expedientes/<id>/`, deducible del identificador del expediente. Se convirtió en columna calculada, previa verificación de que las filas existentes siguieran el patrón; si alguna no lo hubiera hecho, la migración se habría detenido.

5.3 Coherencia entre la factura y su cita

Cuando una factura está asociada a una cita, el paciente y el doctor están guardados en los dos lugares. Nada obligaba a que coincidieran: era posible facturarle a una persona la cita de otra. Las columnas no se eliminaron, porque existen facturas sin cita asociada; se agregó un trigger que valida la coherencia cuando la cita está presente.

5.4 La tasa de ISV no era recuperable

Cada línea de factura tenía un booleano `aplicaISV`, mientras que la factura guardaba `isv15` e `isv18` por separado. No había forma de saber qué tasa se le había aplicado a cada línea, de modo que el impuesto **no podía recalcularse desde el detalle**. Se reemplazó por `tasaISV`, que guarda la tasa efectiva.

5.5 Reglas de negocio que ninguna tabla hacía cumplir

Las tablas `EspecialidadDoctor` y `ServicioEspecialidad` existían pero no gobernaban nada. Al revisar los datos, en **ocho de las nueve citas activas** el doctor asignado no tenía la especialidad que el servicio requiere. El propio ingeniero había descrito para qué debían servir: *«hay servicios odontológicos que requieren una especialidad de un doctor [...] eso es lo que se brinda para poder ver si tenemos un doctor disponible para ese servicio o no»*.

Aplicar la regla directamente habría hecho fallar la migración. Se resolvió en dos pasos: primero se acreditó a cada doctor la especialidad de los servicios que ya venía atendiendo, y recién después se activó la validación. Las doce citas activas cumplen ahora la regla.

5.6 Revisiones que no encontraron problemas

Conviene dejar constancia de lo que se verificó y estaba correcto: las diez facturas cuadran exactamente con la suma de sus líneas; los indicadores de actividad de empleado y usuario no se contradicen en ningún registro; el formato de los correlativos de factura cumple la norma; y `ExpedienteDoctor` **no** es redundante, porque tiene trece asociaciones contra diez derivables de las consultas, es decir tres doctores con acceso concedido antes de haber atendido.

6. Facturación y régimen fiscal

Este capítulo no proviene de la revisión del ingeniero sino del régimen de facturación vigente en Honduras (Acuerdo 481-2017 del SAR y sus reformas).

6.1 El CAI estaba repetido en cada factura

Las diez facturas de la base tenían el mismo CAI copiado idénticamente. Eso es redundancia: el CAI **no es un dato de la factura**, sino del rango de facturación autorizado al que la factura pertenece. Junto a él van el rango de correlativos y la fecha límite de emisión, ninguno de los cuales existía en la base pese a ser obligatorios.

Se creó la tabla `RangoFacturacion`, que agrupa el CAI, los números inicial y final autorizados y la fecha límite. Un trigger impide emitir una factura fuera del rango o con el CAI vencido, que es el bloqueo que la normativa exige. Los correlativos, que ya seguían el formato de dieciséis dígitos, ahora tienen además una restricción que lo verifica.

Importante: el número final del rango y la fecha límite se cargaron con valores provisionales, porque la autorización real del SAR no estaba disponible. Hay que reemplazarlos por los datos verdaderos antes de usar el sistema en producción.

6.2 Requisitos que siguen faltando

Se documentan sin implementar, por decisión de alcance:

Falta	Detalle
RTN del emisor	No existe ninguna tabla con los datos del negocio: razón social, RTN y dirección del punto de emisión. Una búsqueda de «RTN» en toda la base y en el backend no arroja resultados.
RTN del cliente	Obligatorio cuando la compra supera L.100. Las facturas de la base van de L.350 a L.4,019, así que casi todas lo superan. Persona tiene DNI pero no RTN.
Importes gravados	Existen isv15 e isv18 (los impuestos) y los importes exento y exonerado, pero no las bases gravadas de cada tasa.
Estado y tipo de documento	Una factura anulada debe conservarse, no borrarse. Tampoco se contemplan notas de crédito ni de débito.

7. Datos inválidos encontrados

La auditoría no se limitó a la estructura. Al aplicar las restricciones nuevas aparecieron datos que la base venía aceptando sin objeción:

Códigos de verificación que expiraban antes de existir. Los diez registros tenían fecha de expiración ocho días *anterior* a la de creación. La migración los corrige y agrega la restricción que impide que vuelva a ocurrir.

Ocho de nueve citas activas con el doctor equivocado. El doctor asignado no tenía la especialidad que el servicio requiere, como se detalla en 5.5.

Descripciones de otra clínica. Las líneas de factura de prueba mencionan «Consulta dermatológica», «Consulta pediátrica» y «Electrocardiograma», que no corresponden a una clínica odontológica. Es un defecto de los datos semilla, no del esquema, pero conviene limpiarlo antes de presentar el proyecto.

8. Pruebas de comportamiento

Comprobar que una restricción existe no es lo mismo que comprobar que funciona. Se ejecutaron 36 pruebas sobre la base migrada, cada una intentando una operación concreta y verificando si la base la aceptaba o la rechazaba. Todas dieron el resultado esperado.

Operación intentada	Resultado
Doble reserva del mismo doctor a la misma hora	Rechazada
Doble reserva del mismo paciente a la misma hora	Rechazada
Reutilizar el horario de una cita cancelada	Aceptada
Agendar una cita con la recepcionista	Rechazada
Cita con un doctor sin la especialidad del servicio	Rechazada
Cita con un doctor que sí tiene la especialidad	Aceptada
Guardar «mañana» como fecha de una cita	Rechazada
DNI duplicado	Rechazada
Persona sin primer apellido	Rechazada
Correo repetido cambiando mayúsculas	Rechazada
Precio negativo en un servicio	Rechazada
Cantidad cero en una línea de factura	Rechazada
Facturar una cita pendiente	Rechazada
Facturar con un paciente distinto al de la cita	Rechazada
Facturar correctamente una cita completada	Aceptada
Número de factura fuera del rango autorizado	Rechazada
Emitir con el CAI vencido	Rechazada
Número de factura con formato inválido	Rechazada
Tasa de ISV distinta de 0, 15 o 18 por ciento	Rechazada
Segundo token vivo del mismo tipo para un usuario	Rechazada
Token que expira antes de crearse	Rechazada
Recordatorio repetido para la misma cita	Rechazada
Varias cancelaciones para la misma cita	Aceptada
Segundo código de verificación para el mismo usuario	Aceptada
updatedAt se actualiza solo al modificar una fila	Correcto
logout queda nulo en una sesión abierta	Correcto
Borrar una factura elimina sus líneas	Correcto

Se omiten nueve pruebas auxiliares de preparación de datos.

9. Mejoras pendientes

Se dejan documentadas y sin implementar, por decisión de alcance. Ninguna es un defecto introducido por este trabajo: son limitaciones del modelo original que exceden lo acordado.

Pendiente	Por qué no se hizo
Entidad Paciente propia	Cita y Factura apuntan directamente a Persona, mientras el empleado sí tiene tabla propia. Es asimétrico y no permite distinguir quién es paciente. Obliga a reescribir todas las consultas de citas y facturas.
Bandera de cambio de contraseña	Pedida por el ingeniero. No fue autorizada en esta ronda. El backend hace referencia a tres columnas inexistentes para esto.
Horario de atención por doctor	Nada impide agendar una cita a las tres de la mañana. Los datos actuales van de 08:00 a 16:00 por casualidad, no porque la base lo exija.
Registro de accesos para clientes	La tabla Logs apunta a Empleado, no a User, así que los usuarios con rol cliente no dejan rastro.
Unificar el tipo de fechaCancelacion	Es la única columna con zona horaria entre treinta y una. Convertirla desplazaría los valores según la zona del servidor.
Datos fiscales de la empresa	Razón social, RTN y dirección del punto de emisión, más el RTN del cliente y las bases gravadas.

10. Archivos entregados

Archivo	Contenido
003_pulido_esquema.sql	Primera migración: nombres, tipos de datos, catálogos Rol y Puesto, integridad e índices.
003_rollback.sql	Reversión completa de la migración 003, verificada.
004_correcciones_ingeniero.sql	Segunda migración: permisos, auditoría, nombres mnemónicos, tokens, recordatorios, facturación y reglas de negocio.
esquema_final.sql	Crea la base desde cero con todo aplicado. Es la documentación del modelo.
CAMBIOS_Y_IMPACTO.md	Guía técnica de los cambios y del trabajo pendiente en backend y frontend.

Advertencia sobre la aplicación de las migraciones. Ambas se ejecutan dentro de una transacción: si algo falla, no se aplica ningún cambio. Esa protección solo funciona si el script se ejecuta completo. En DataGrip, el modo de transacción «Auto» confirma cada sentencia por separado y anula esa garantía; conviene usar «Manual» o ejecutar por línea de comandos.